

IEORE4004

Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research



Outline

- Maximum Flow Problem
 - LP Formulation
 - Ford-Fulkerson Algorithm
- Transportation Problem
 - LP Formulation
 - Balanced TPs
 - Balancing Unbalanced TPs
 - Transshipment Problem
 - Assignment Problem
 - The Hungarian Method
 - Minimum-Cost Network Flow Problems

Maximum Flow Problem

- LP Formulation
- Ford-Fulkerson Algorithm

The Maximum Flow Problem

This problem seeks the greatest possible flow from a starting point (called the source) to a terminal point (called the sink) in a network, respecting the **capacity constraints** of its edges.



Many situations can be modeled by a network in which the arcs may be thought of as having a capacity that limits the quantity of a product that may be shipped through the arc.

Maximum Flow Problem Use Cases



Transportation and Logistics: can model the maximum number of goods that can be transported from a factory to a distribution center through various routes with different capacities

Telecommunications: can represent the maximum data that can be sent from one node to another in a network

Project Scheduling: Especially in cases where there are dependencies between tasks, the flow can help determine the maximum set of tasks that can be undertaken simultaneously

The Maximum Flow Problem Formulation

The input is a directed graph $G = (V, E)$ with two special vertices, a "source" $s \in V$, and a "sink" $t \in V$.

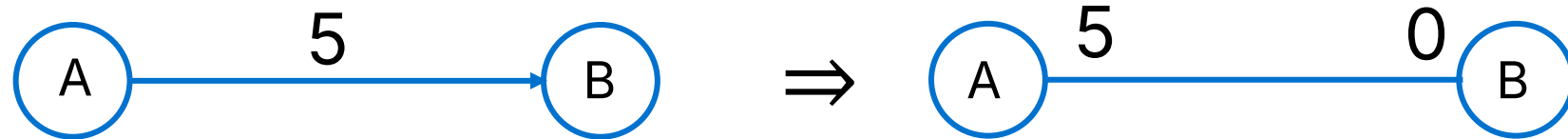
For every node $v \in \{s, t\}$, the incoming flow to v must be equal to the outgoing flow from v .

$$\begin{aligned} \max_{\mathbf{f}} \quad & \sum_{v:s \rightarrow v} f_{sv} - \sum_{u:u \rightarrow s} f_{us} \\ \text{s.t.} \quad & \sum_{u:u \rightarrow v} f_{uv} = \sum_{w:v \rightarrow w} f_{vw}, \quad \forall v \neq s, t \quad (\text{flow conservation}) \\ & 0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E \quad (\text{capacity constraints}) \end{aligned}$$

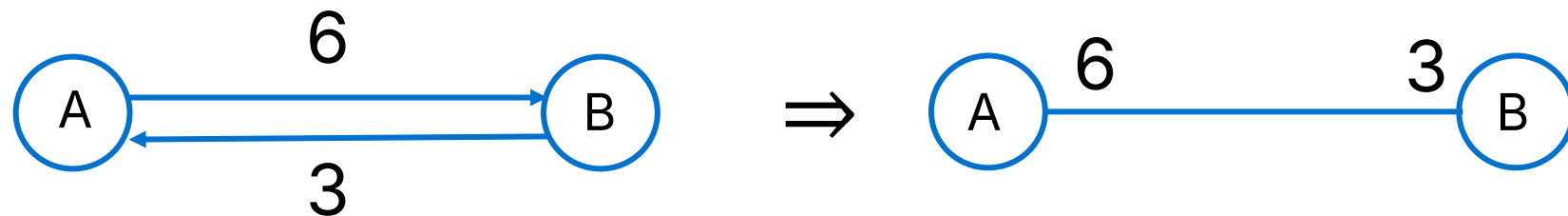
Residual Network

Residual network is the network which shows the **residual capacities** of the arcs (unused capacities) after some flows have been assigned through the original network.

At the beginning, a directed arc (capacity in one direction) is converted to an arc with both feasible directions, where the opposite direction has zero capacity.

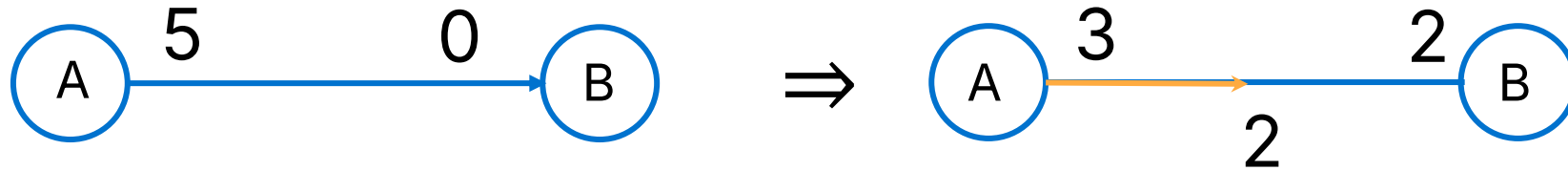


Similarly,



Residual Network

When a flow in arc $A \rightarrow B$ is assigned equal to 2 units, the residual capacities become,



An **augmenting path** is a path from the **source** to the **sink** in the residual network such that every arc on this path has strictly positive residual capacity. The minimum of these capacities is called **residual capacity of the augmenting path**.

Ford-Fulkerson Algorithm

The Ford-Fulkerson Algorithm In A Nutshell

Step 1: Identify an augmenting path through the network and denote its residual capacity by c^* . If no augmenting path exists, go to Step 3.

The Ford-Fulkerson Algorithm In A Nutshell

Step 1: Identify an augmenting path through the network and denote its residual capacity by c^* . If no augmenting path exists, go to Step 3.

Step 2: Assign a flow equal to c^* to the identified augmenting path. For each arc of this path, directed from source to sink, reduce its residual capacity by c^* and increase the residual capacity by c^* for all arcs in the opposite direction. Go to Step 1.

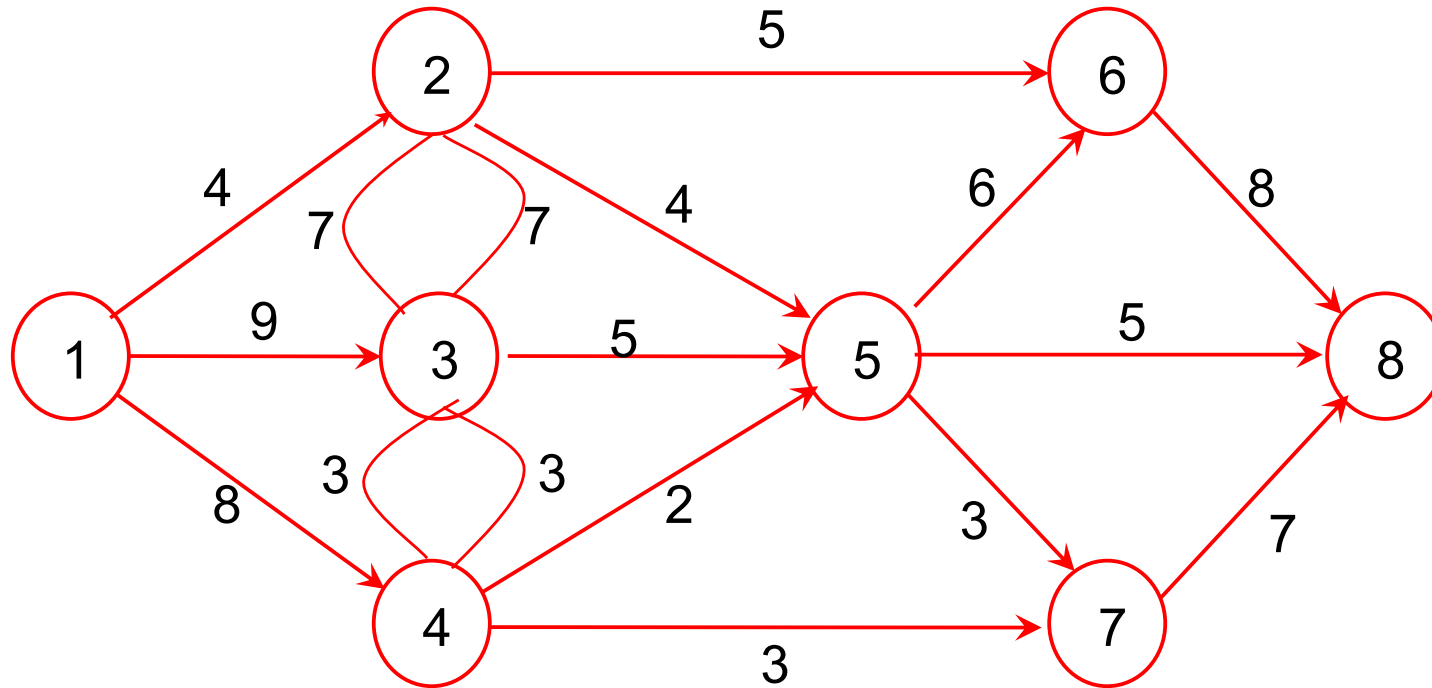
The Ford-Fulkerson Algorithm In A Nutshell

Step 1: Identify an augmenting path through the network and denote its residual capacity by c^* . If no augmenting path exists, go to Step 3.

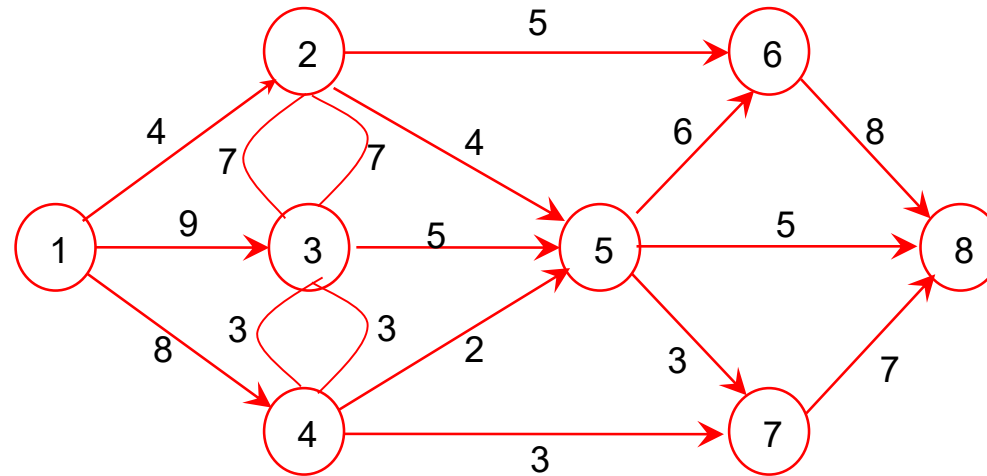
Step 2: Assign a flow equal to c^* to the identified augmenting path. For each arc of this path, directed from source to sink, reduce its residual capacity by c^* and increase the residual capacity by c^* for all arcs in the opposite direction. Go to Step 1.

Step 3: Stop; the net flows already assigned constitute an optimal flow pattern.

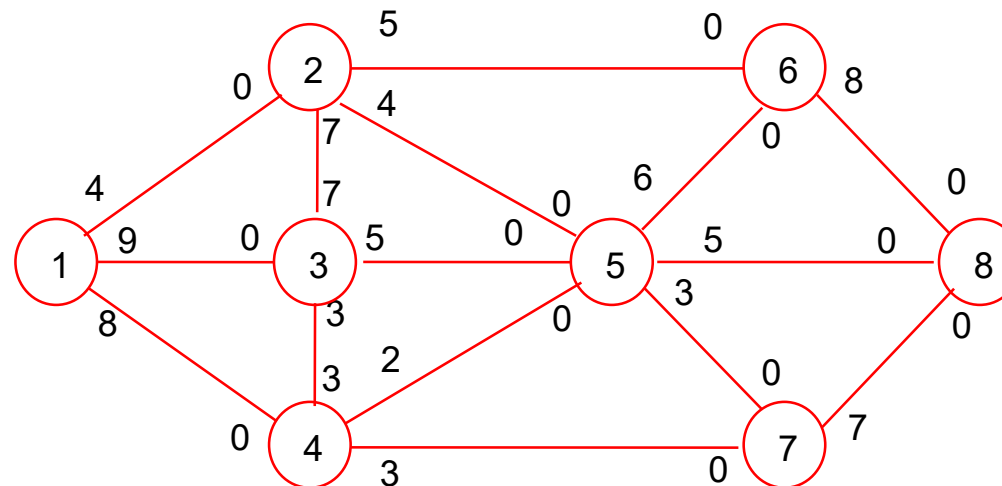
Example: The Ford-Fulkerson Algorithm



Example: The Ford-Fulkerson Algorithm



Convert the arcs to bi-directional with capacities on both directions:

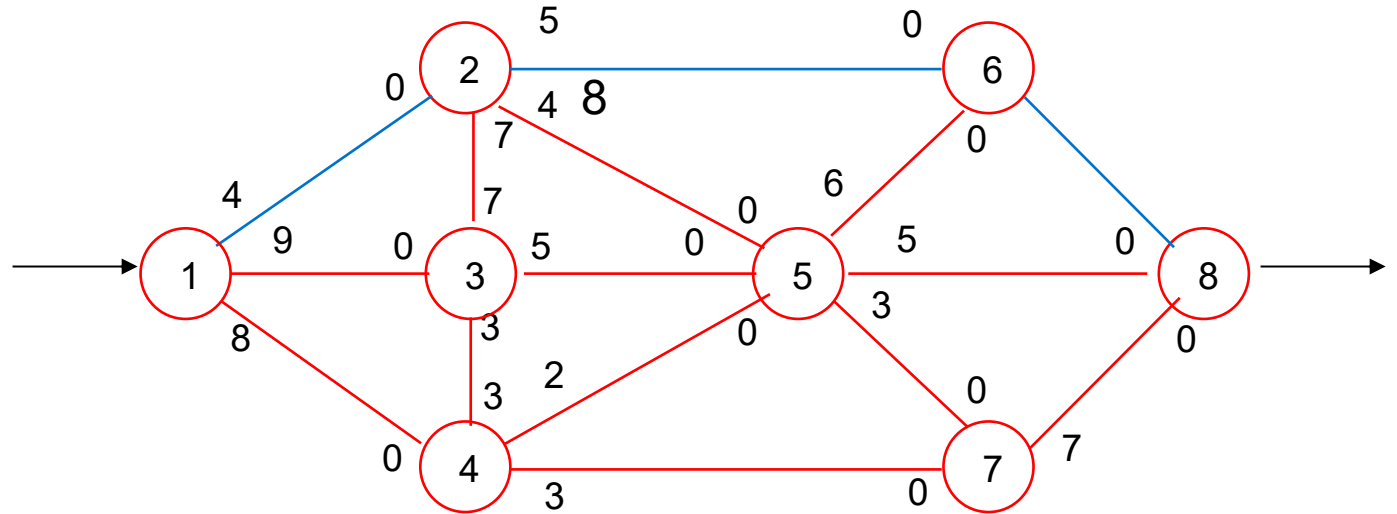


Example: The Ford-Fulkerson Algorithm

Iteration 1

Augmenting path **(1→2→6→8)**
with a flow equal to 4

Current flow value **$z = 4$**

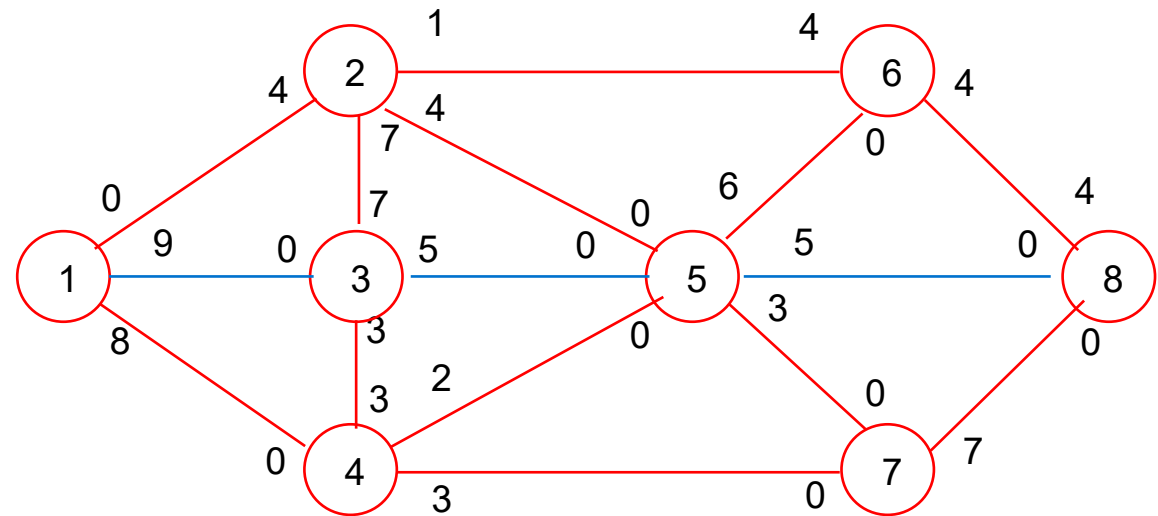


Iteration 2

Update the residual capacity
considering iteration 1's augmented
path.

Then, augmenting path **(1→3→5→8)**
will be used, with a flow 5

$z = z + 5 \Rightarrow 9$

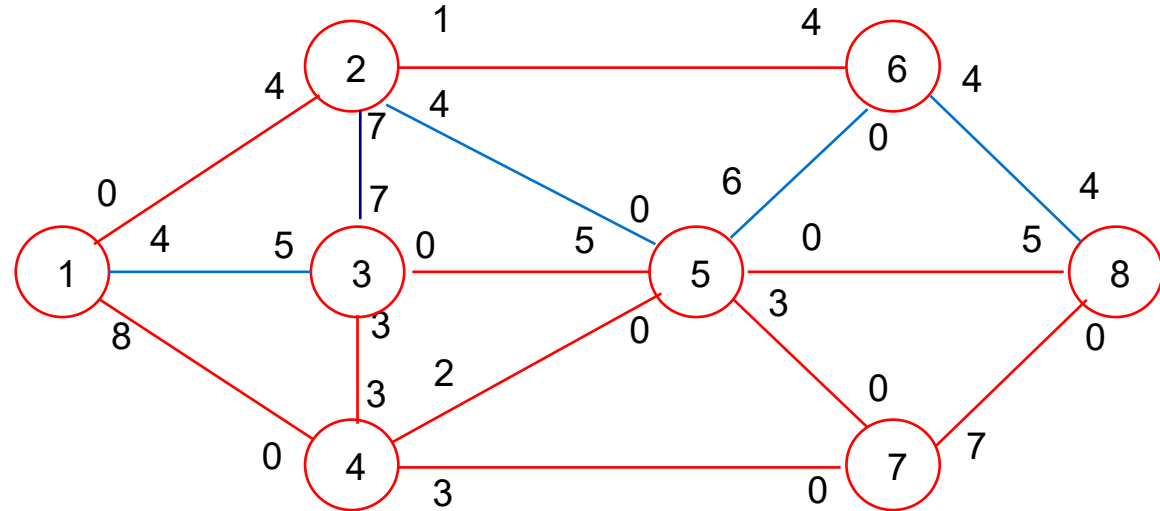


Example: The Ford-Fulkerson Algorithm

Iteration 3

Path: $(1 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 6 \rightarrow 8)$

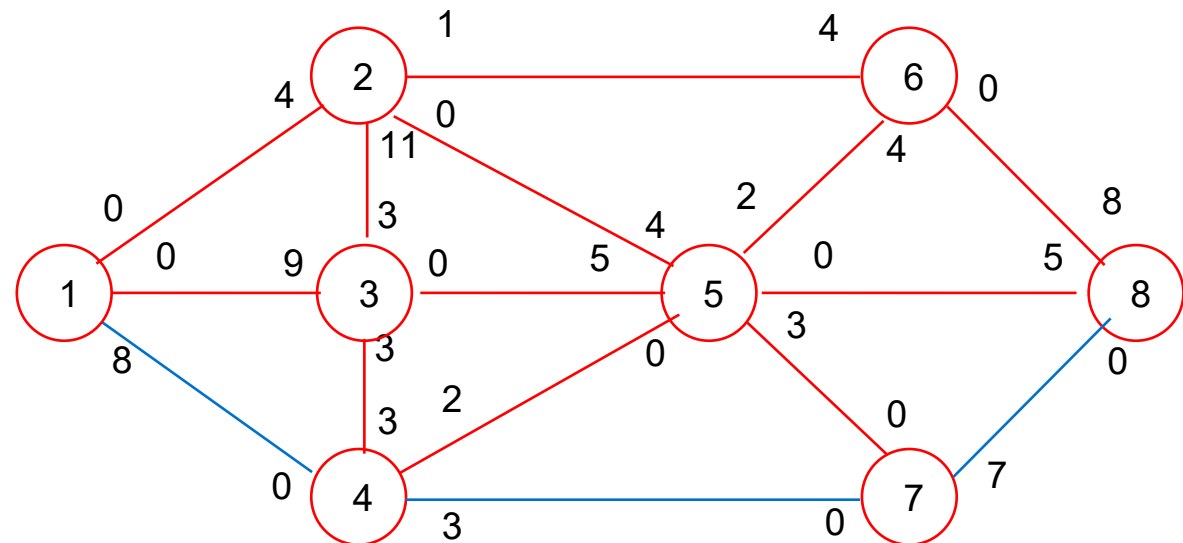
$$z = 9 + 4 = 13$$



Iteration 4

Path: $(1 \rightarrow 4 \rightarrow 7 \rightarrow 8)$

$$z = 13 + 3 = 16$$



Iteration 5

$$z = 16 + 2 = 18$$


$(1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 6 \rightarrow 5 \rightarrow 7 \rightarrow 8)$

$$z = 18 + 1 = 19$$





Example: The Pipeline Problem

Sunco Oil wants to ship the maximum possible amount of oil (per hour) via pipeline from node so to node si . On its way from node so to node si , oil must pass through some or all of stations 1, 2, and 3. The various arcs represent pipelines of different diameters. The maximum number of barrels of oil (millions of barrels per hour) that can be pumped through each arc is shown in Table. Formulate an LP that can be used to determine the maximum number of barrels of oil per hour that can be sent from so to si .

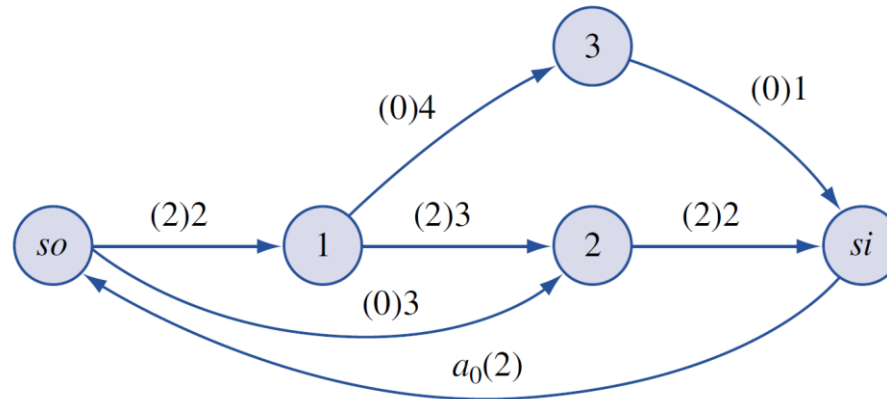
Arc	Capacity
$(so, 1)$	2
$(so, 2)$	3
$(1, 2)$	3
$(1, 3)$	4
$(3, si)$	1
$(2, si)$	2



Example: Maximum Flow Problem

Node so is called the source node because oil flows out of it but no oil flows into it, and node si is called the sink node.

Let, x_{ij} millions of barrels of oil per hour that will pass through arc (i, j) of pipeline.



For a flow to be feasible, it must have two characteristics:

1. $0 < \text{flow through each arc} < \text{arc capacity}$
2. $\text{Flow into node } i = \text{flow out of node } i$



Example: Maximum Flow Problem

$$\max z = x_0$$

$$\begin{aligned} \text{s.t.} \quad & x_{so,1} \leq 2 && \text{(Arc capacity constraints)} \\ & x_{so,2} \leq 3 \\ & x_{12} \leq 3 \\ & x_{2,si} \leq 2 \\ & x_{13} \leq 4 \\ & x_{3,si} \leq 1 \\ & x_0 = x_{so,1} + x_{so,2} && \text{(Node } so \text{ flow constraint)} \\ & x_{so,1} = x_{12} + x_{13} && \text{(Node 1 flow constraint)} \\ & x_{so,2} + x_{12} = x_{2,si} && \text{(Node 2 flow constraint)} \\ & x_{13} = x_{3,si} && \text{(Node 3 flow constraint)} \\ & x_{3,si} + x_{2,si} = x_0 && \text{(Node } si \text{ flow constraint)} \\ & x_{ij} \geq 0 \end{aligned}$$



Example: The Matchmaking Problem

Five lead and five follows are at a dance. The goal of the matchmaker is to match each lead with a follow in a way that maximizes the number of people who are matched with compatible mates.

Table below describes the compatibility of the entertainers.

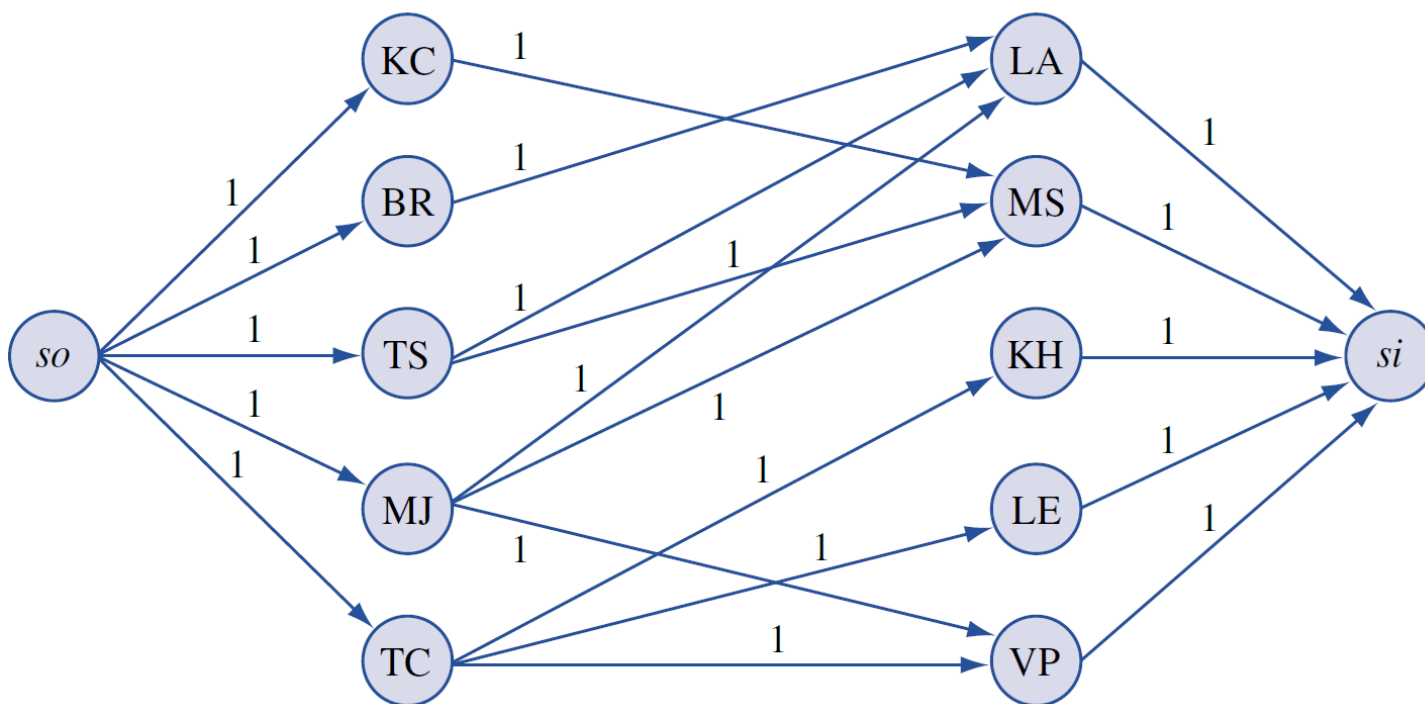
Draw a network that makes it possible to represent the problem of maximizing the number of compatible pairings as a maximum-flow problem.

	Luca Amato	Ming Su	Karan Handa	Li En	Valeria Pellegrini
Kareem Chams	-	C	-	-	-
Bai Ru	C	-	-	-	-
Tao Shen	C	C	-	-	-
Manoj Jain	C	C	-	-	C
Tian Chao	-	-	C	C	C



Matchmaking Problem

The maximum flow in this network is the number of compatible couples that can be created by the matchmaker!



If the matchmaker pairs:

- KC and MS,
 - BR and LA,
 - MJ and VP, and
 - TC and KH,
- a flow of 4 from source to sink would be obtained.

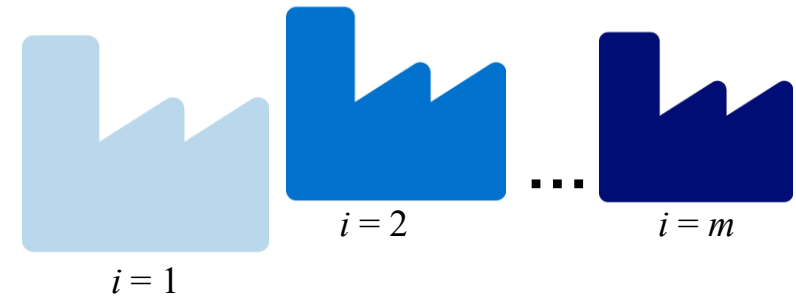
Transportation Problem

- LP Formulation
- Assignment Problem
- Transshipment Problem
- Minimum-Cost Network Flow Problems

Transportation Problem

In general, a transportation problem is specified by the following information:

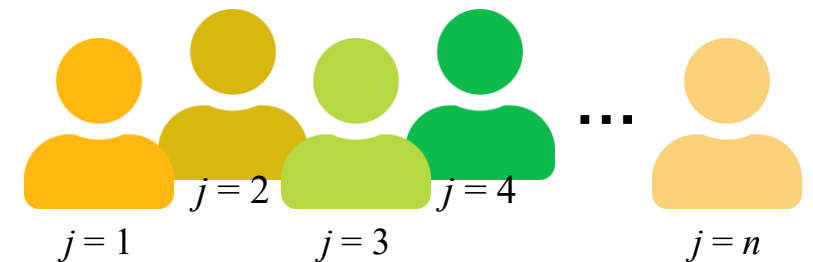
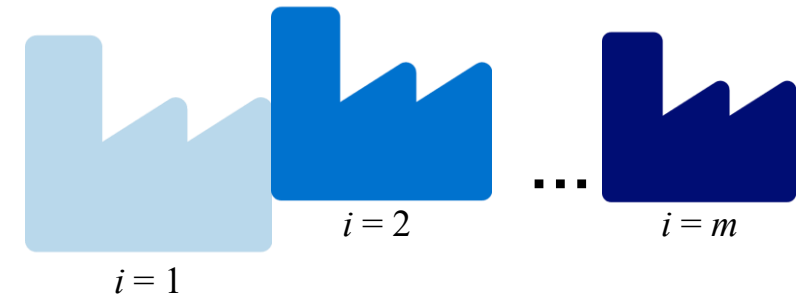
1. A set of m supply points from which a good is shipped. Supply point i can supply at most s_i units.



Transportation Problem

In general, a transportation problem is specified by the following information:

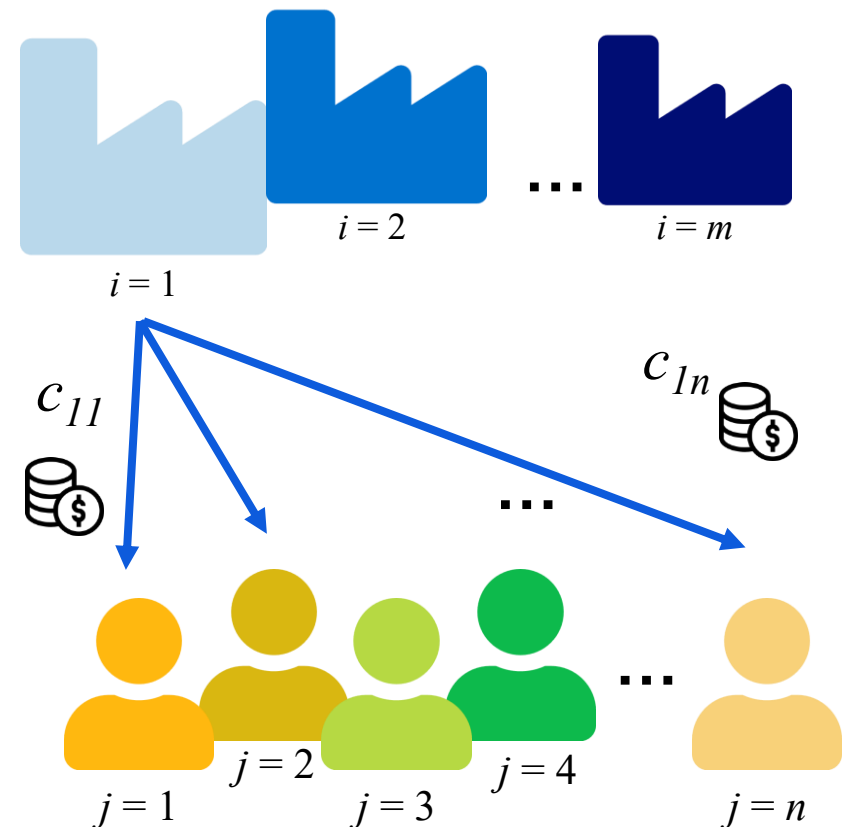
1. A set of m supply points from which a good is shipped. Supply point i can supply at most s_i units.
2. A set of n demand points to which the good is shipped. Demand point j must receive at least d_j units of the shipped good.



Transportation Problem

In general, a transportation problem is specified by the following information:

1. A set of m supply points from which a good is shipped. Supply point i can supply at most s_i units.
2. A set of n demand points to which the good is shipped. Demand point j must receive at least d_j units of the shipped good.
3. Each unit produced at supply point i and shipped to demand point j incurs a variable cost of c_{ij}

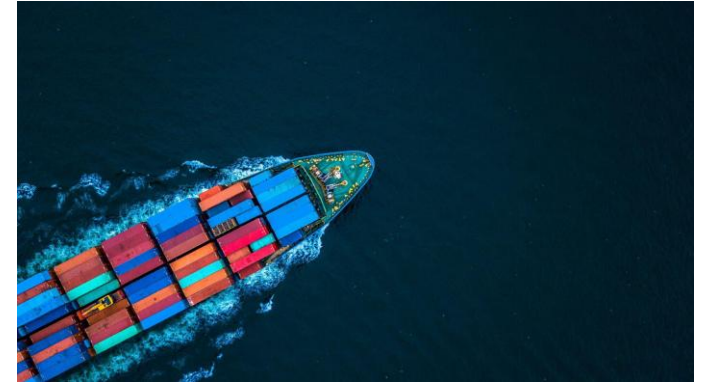


Transportation Problem Real Life Examples

Shipping Goods from Warehouses to Stores: Deciding how many units of a product to ship from each warehouse to each store to minimize transportation costs while meeting store demand

Fuel Distribution: Oil companies need to supply gasoline to numerous gas stations from their refineries or storage units. They must determine the optimal way to distribute the gasoline while considering costs and ensuring each station's demand is met.

Relocation of Refugees: In humanitarian logistics, one might need to transport refugees from several camps to different resettlement areas, considering the capacity of each resettlement area, and the costs associated with each possible route.



Transportation Problem LP Formulation

Let x_{ij} be the number of units shipped from supply point i to demand point j .
Then the general formulation of a transportation problem is:

$$\begin{aligned} \min \quad & \sum_{i=1}^{i=m} \sum_{j=1}^{j=n} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^{j=n} x_{ij} \leq s_i \quad (i = 1, 2, \dots, m) && \text{(Supply constraints)} \\ & \sum_{i=1}^{i=m} x_{ij} \geq d_j \quad (j = 1, 2, \dots, n) && \text{(Demand constraints)} \\ & x_{ij} \geq 0 \quad (i = 1, 2, \dots, m; j = 1, 2, \dots, n) \end{aligned}$$

Balanced Transportation Problem

If the total supply equals total demand, then the problem is a **balanced transportation problem**

$$\sum_{i=1}^{i=m} s_i = \sum_{j=1}^{j=n} d_j$$

The diagram illustrates the balanced transportation problem. It starts with a mathematical equation: $\sum_{i=1}^{i=m} s_i = \sum_{j=1}^{j=n} d_j$. To the right of the equation is a sequence of icons connected by arrows. The first icon is a factory, representing supply. An arrow points to an equals sign. Another arrow points to a warehouse icon, representing demand. A final arrow points to a group of people, also representing demand. This visualizes the concept that total supply equals total demand.



In a balanced transportation problem,
all the constraints must be binding

Balanced Transportation Problem LP Formulation

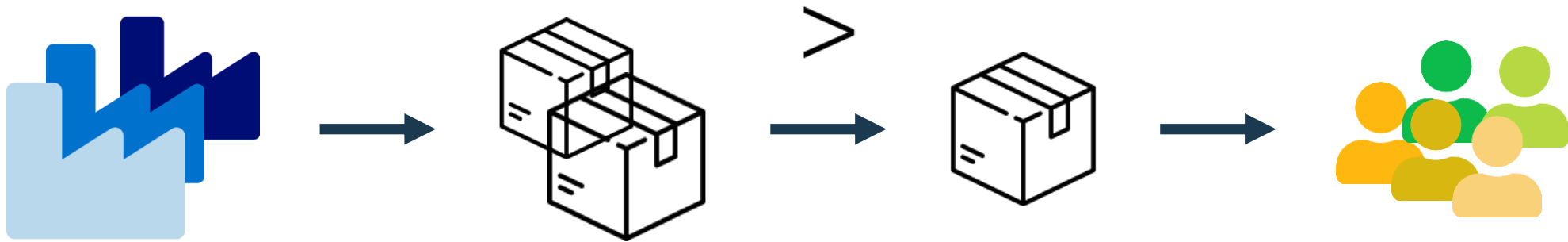
$$\begin{aligned} \min \quad & \sum_{i=1}^{i=m} \sum_{j=1}^{j=n} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^{j=n} x_{ij} \boxed{=} s_i \quad (i = 1, 2, \dots, m) && \text{(Supply constraints)} \\ & \sum_{i=1}^{i=m} x_{ij} \boxed{=} d_j \quad (j = 1, 2, \dots, n) && \text{(Demand constraints)} \\ & x_{ij} \geq 0 \quad (i = 1, 2, \dots, m; j = 1, 2, \dots, n) \end{aligned}$$

Binding!

Balancing A Transportation Problem (TP)

If **total supply exceeds total demand**, we can balance a TP by creating a **dummy demand point** that has a demand equal to the amount of excess supply.

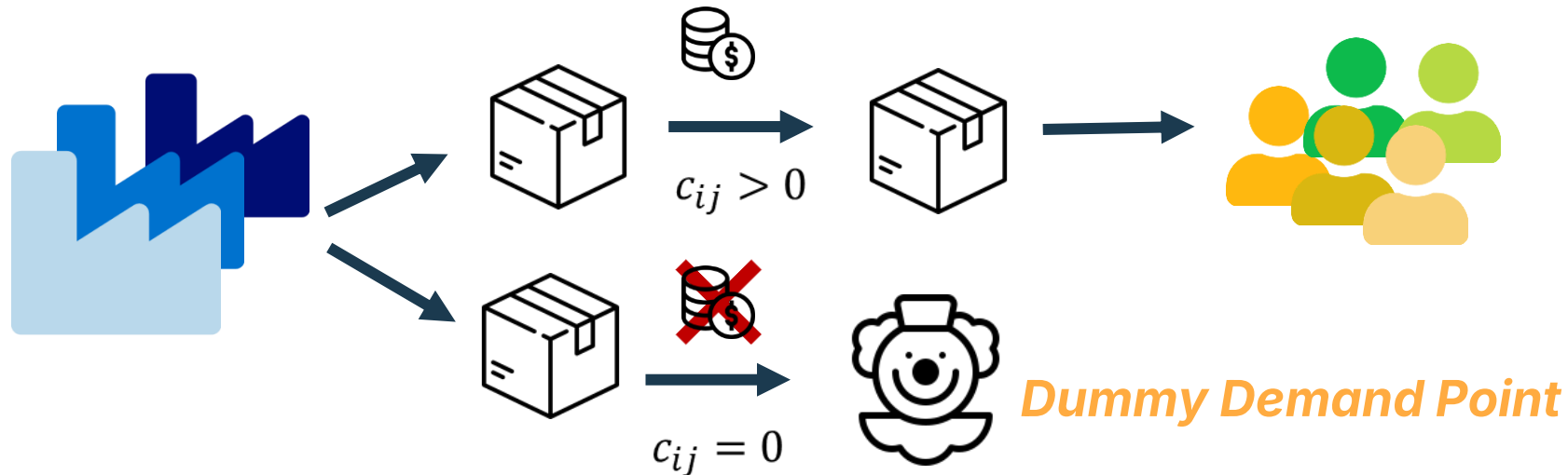
- Because shipments to the dummy demand point are not real shipments, they are assigned a cost of zero.
- Shipments to the dummy demand point indicate unused supply capacity.



Balancing A Transportation Problem (TP)

If **total supply exceeds total demand**, we can balance a TP by creating a **dummy demand point** that has a demand equal to the amount of excess supply.

- Because shipments to the dummy demand point are not real shipments, they are assigned a cost of zero.
- Shipments to the dummy demand point indicate unused supply capacity.



Example: Balancing A Transportation Problem

Two reservoirs are available to supply the water needs of three cities. Each reservoir can supply up to **50 million gallons** of water per day. Each city would like to receive **40 million gallons** per day. For each million gallons per day of unmet demand, there is a penalty. At **city 1**, the **penalty is \$20**; at **city 2**, the **penalty is \$22**; and at **city 3**, the **penalty is \$23**. The cost of transporting **1 million gallons of water** from each reservoir to each city is shown in Table below.

Shipping Costs for Reservoir

From	To		
	City 1	City 2	City 3
Reservoir 1	\$7	\$8	\$10
Reservoir 2	\$9	\$7	\$8



Can you formulate a balanced transportation problem that can be used to minimize the sum of shortage and transport costs?

Example: Balancing A Transportation Problem

Step 1: Check supply and demand

Daily supply = $50 + 50 = 100$ million gallons per day

Daily demand = $40 + 40 + 40 = 120$ million gallons per day

Step 2: Balance the problem by adding a dummy (or shortage) variable
Add a **dummy supply point** with supply of $120 - 100 = 20$ mil. gallons/day

The transportation costs from dummy reservoir D to the cities are equivalent to the penalty costs of shortages:

$$C_{D1} = \$20, \quad C_{D2} = \$22, \quad C_{D3} = \$23$$

Example: Balancing A Transportation Problem

Step 3: Formulate your LP as it's a balanced TP and solve

Let x_{ij} be million gallons transported from reservoir i to city j .

Minimize $Z = 7x_{11} + 8x_{12} + 10x_{13} + 9x_{21} + 7x_{22} + 9x_{23} + 20x_{D1} + 22x_{D2} + 23x_{D3}$

$$x_{11} + x_{12} + x_{13} \leq 50$$

(Supply from reservoir 1)

$$x_{21} + x_{22} + x_{23} \leq 50$$

(Supply from reservoir 2)

$$x_{D1} + x_{D2} + x_{D3} \leq 120$$

(Maximum shortage)

$$x_{11} + x_{21} + x_{D1} = 40$$

(Demand for city 1)

$$x_{12} + x_{22} + x_{D2} = 40$$

(Demand for city 2)

$$x_{13} + x_{23} + x_{D3} = 40$$

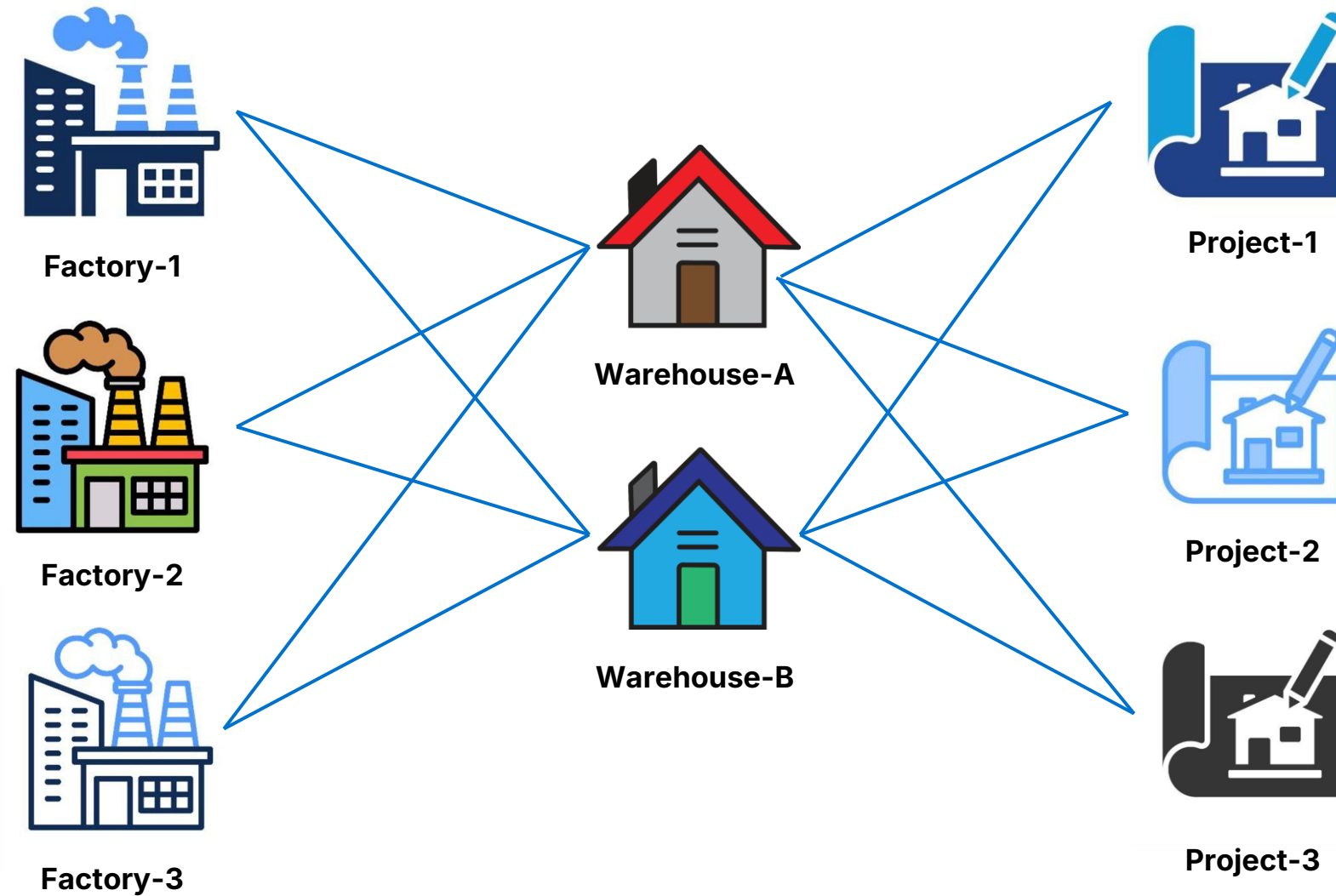
(Demand for city 3)

$$x_{ij} \geq 0, \forall i, j$$

(Nonnegativity constraints)

Transshipment Problem

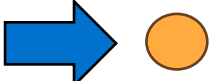
Transshipment Problem



Transshipment Problem

A transportation problem allows only shipments that go directly from a supply point to a demand point. In transshipment problem, goods can be transshipped on their journey from a supply point to a demand point.

The optimal solution to a transshipment problem can be found by solving a transportation problem (*not a surprise*)

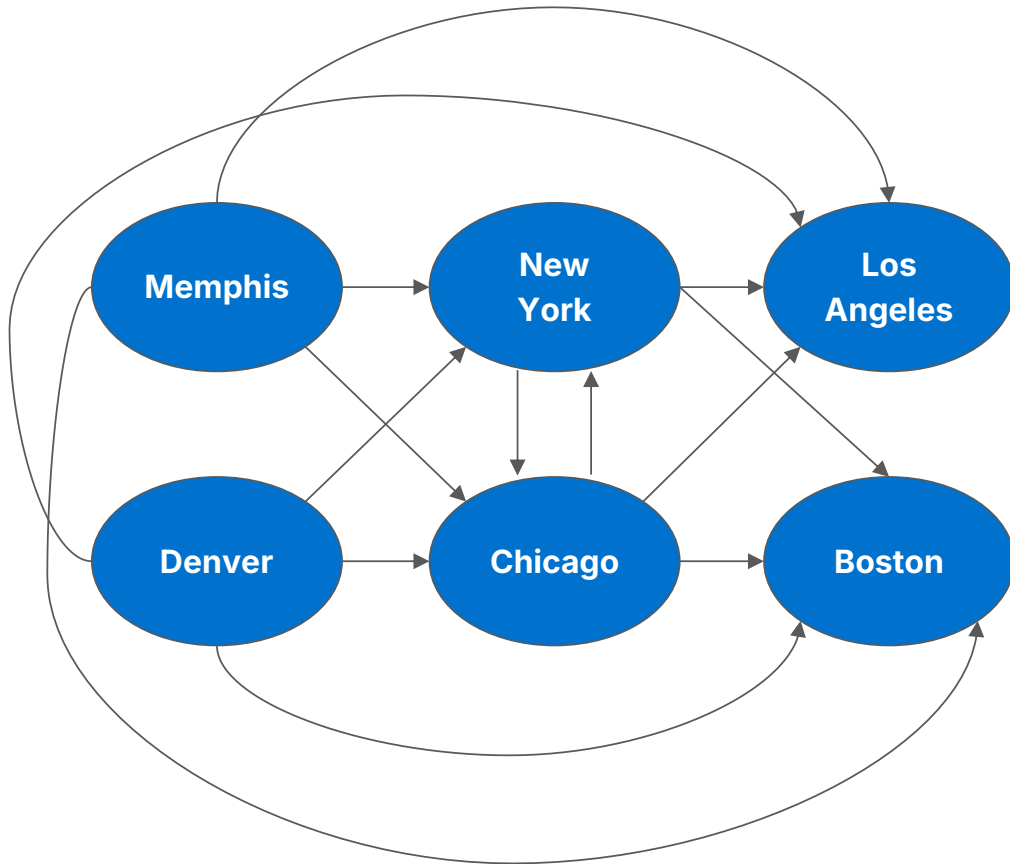
 Supply point:	can send goods to another point but cannot receive goods from any other point
 Demand point:	can receive goods from other points but cannot send goods to any other point
 Transshipment point:	can both receive goods from other points and send goods to other points

Transshipment Problem

Example: Widgetco manufactures widgets at two factories, one in Memphis and one in Denver. The Memphis factory can produce as many as 150 widgets per day, and the Denver factory can produce as many as 200 widgets per day. Widgets are shipped by air to customers in Los Angeles and Boston. The customers in each city require 130 widgets per day. Because of the deregulation of airfares, Widgetco believes that it may be cheaper to first fly some widgets to New York or Chicago and then fly them to their final destinations. The costs of flying a widget are shown in Table below. Widgetco wants to minimize the total cost of shipping the required widgets to its customers.

	To (\$)					
From	Memphis	Denver	N.Y.	Chicago	L.A.	Boston
Memphis	0	-	8	13	25	28
Denver	-	0	15	12	26	25
N.Y.	-	-	0	6	16	17
Chicago	-	-	6	0	14	16
L.A.	-	-	-	-	0	-
Boston	-	-	-	-	-	0

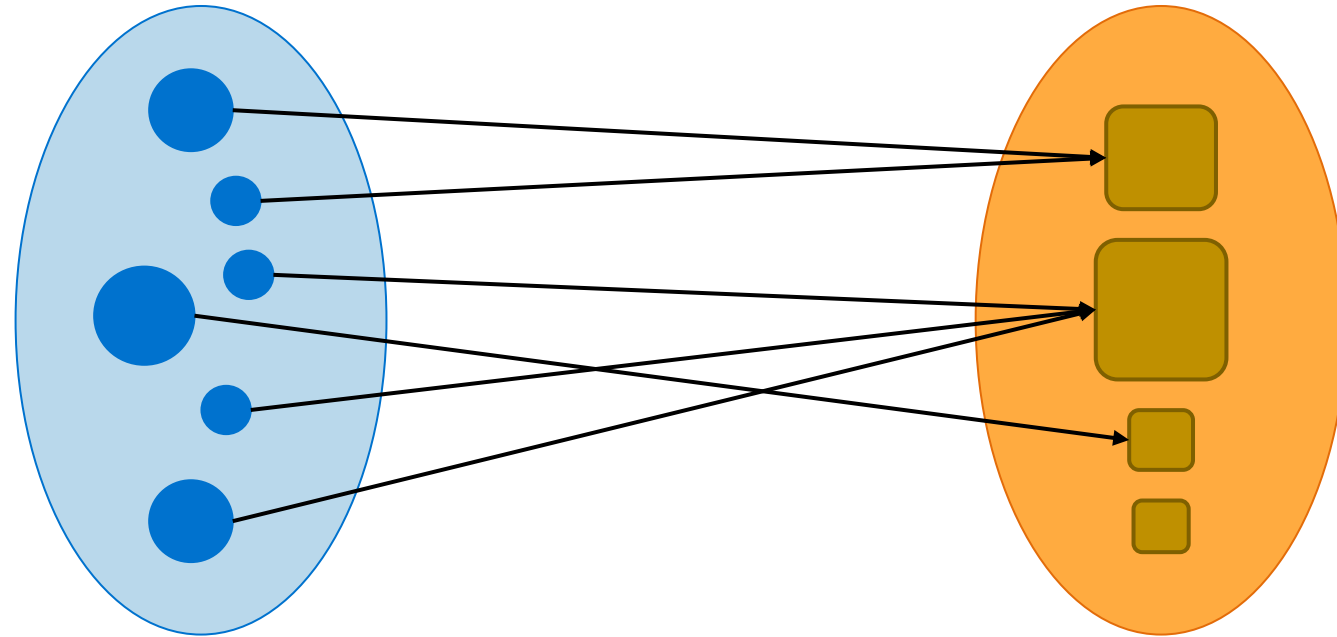
Transshipment Problem LP Solution



	To (\$)					
From	Memphis	Denver	N.Y.	Chicago	L.A.	Boston
Memphis	0	-	8	13	25	28
Denver	-	0	15	12	26	25
N.Y.	-	-	0	6	16	17
Chicago	-	-	6	0	14	16
L.A.	-	-	-	-	0	-
Boston	-	-	-	-	-	0

Assignment Problem

Assignment Problem



Tasks, Jobs,
Facilities,
Equipments, ...

Workers, Processors,
Locations, Resources,
...

Assignment Problem

- ! An assignment problem is a balanced transportation problem in which **all supplies and demands equal 1**

Example: A manufacturer has four machines and four jobs to be completed. Each machine must be assigned to complete one job. The time required to set up each machine for completing each job is shown in Table below. Manufacturer wants to minimize the total setup time needed to complete the four jobs.

	Time (Hours)			
Machine	Job 1	Job 2	Job 3	Job 4
1	14	5	8	7
2	2	12	6	5
3	7	8	3	9
4	2	4	6	10

? Use linear programming to solve this problem.

A Manufacturing Problem

$x_{ij} = 1$ if machine i is assigned to meet the demands of job j

$x_{ij} = 0$ if machine i is not assigned to meet the demands of job j

$$\begin{aligned} \min z = & 14x_{11} + 5x_{12} + 8x_{13} + 7x_{14} + 2x_{21} + 12x_{22} + 6x_{23} + 5x_{24} \\ & + 7x_{31} + 8x_{32} + 3x_{33} + 9x_{34} + 2x_{41} + 4x_{42} + 6x_{43} + 10x_{44} \end{aligned}$$

$$\text{s.t.} \quad x_{11} + x_{12} + x_{13} + x_{14} = 1 \quad (\text{machine constraints})$$

$$x_{21} + x_{22} + x_{23} + x_{24} = 1$$

$$x_{31} + x_{32} + x_{33} + x_{34} = 1$$

$$x_{41} + x_{42} + x_{43} + x_{44} = 1$$

$$x_{11} + x_{21} + x_{31} + x_{41} = 1 \quad (\text{job constraints})$$

$$x_{12} + x_{22} + x_{32} + x_{42} = 1$$

$$x_{13} + x_{23} + x_{33} + x_{43} = 1$$

$$x_{14} + x_{24} + x_{34} + x_{44} = 1$$

$$x_{ij} = 0 \quad \text{or} \quad x_{ij} = 1$$

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

Step 2: Matrix Reduction

- For rows and columns: Subtract smallest element.

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

Step 2: Matrix Reduction

- For rows and columns: Subtract smallest element.

Step 3: Use minimum lines to cover zeros (horizontal/vertical).

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

Step 2: Matrix Reduction

- For rows and columns: Subtract smallest element.

Step 3: Use minimum lines to cover zeros (horizontal/vertical).

Step 4: If n lines are used, go to Step 5. Else, Step 6.

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

Step 2: Matrix Reduction

- For rows and columns: Subtract smallest element.

Step 3: Use minimum lines to cover zeros (horizontal/vertical).

Step 4: If n lines are used, go to Step 5. Else, Step 6.

Step 5: Assign Tasks using zeros

The Hungarian Method

Step 1: Create a $n \times n$ Matrix (Add dummy rows/columns if non-square.)

Step 2: Matrix Reduction

- For rows and columns: Subtract smallest element.

Step 3: Use minimum lines to cover zeros (horizontal/vertical).

Step 4: If n lines are used, go to Step 5. Else, Step 6.

Step 5: Assign Tasks using zeros

Step 6: Adjust Matrix

- Deduct smallest non-covered value from non-covered elements.
- Add same value to intersections of lines.
- Repeat from Step 3.

Manufacturing Problem - Using Hungarian Method

Step 1: Create Matrix

	Time (Hours)			
Machine	Job 1	Job 2	Job 3	Job 4
1	14	5	8	7
2	2	12	6	5
3	7	8	3	9
4	2	4	6	10

14	5	8	7
2	12	6	5
7	8	3	9
2	4	6	10

Manufacturing Problem - Using Hungarian Method

Step 2: Matrix Reduction

Row Minimum

14	5	8	7
2	12	6	5
7	8	3	9
2	4	6	10

5

2

3

2

9	0	3	2
0	10	4	3
4	5	0	6
0	2	4	8

Column Minimum

0

0

2

Manufacturing Problem - Using Hungarian Method

Step 3: Use minimum lines to cover zeros

! Number of rows $\neq$ Number of lines
covering zeros

Skip to Step 6

9	0	3	0
0	10	4	1
4	5	0	4
0	2	4	6

Manufacturing Problem - Using Hungarian Method

Step 3: Use minimum lines to cover zeros

! Number of rows $\neq$ Number of lines
covering zeros

Skip to Step 6

Step 6: Adjust Matrix

- *Deduct smallest non-covered value from non-covered elements.*
- *Add same value to intersections of lines*

Step 6: Adjust Matrix

If n lines are used, it is optimal!

The optimal assignment is:

$$x_{12} = 1, x_{24} = 1, x_{33} = 1, \text{ and } x_{41} = 1$$

9	0	3	0
0	10	4	1
4	5	0	4
0	2	4	6

10	0	3	0
0	9	3	0
5	5	0	4
0	1	3	5

Minimum-Cost Network Flow Problems

Minimum-Cost Network Flow Problems

The transportation, assignment, transshipment, shortest-path, and maximum flow problems are all special cases of the **minimum-cost network flow problem** (MCNFP)

To define an MCNFP, let

- x_{ij} : number of units of flow sent from node i to node j through $arc(i, j)$
- b_i : net supply (outflow-inflow) at node i
- c_{ij} : cost of transporting 1 unit of flow from node i to node j via $arc(i, j)$
- L_{ij} : lower bound on flow through $arc(i, j)$
(if there is no lower bound, let $L_{ij} = 0$)
- U_{ij} : upper bound on flow through $arc(i, j)$
(if there is no upper bound, let $U_{ij} = \infty$)

Minimum-Cost Network Flow Problems

Then the **MCNFP** may be written as an LP:

$$\begin{aligned} \min \quad & \sum_{\text{all arcs}} c_{ij} x_{ij} && \text{Cost} \\ \text{s.t.} \quad & \sum_j x_{ij} - \sum_k x_{ki} = b_i && \text{Net supply} \\ & L_{ij} \leq x_{ij} \leq U_{ij} && \text{Lower bound on flow, Upper bound on flow} \end{aligned}$$

(for each node i in the network)

(for each arc in the network)

Minimum-Cost Network Flow Problems

Then the **MCNFP** may be written as an LP:

$$\min \sum_{\text{all arcs}} c_{ij}x_{ij}$$

$$\text{s.t.} \quad \sum_j x_{ij} - \sum_k x_{ki} = b_i \quad (\text{for each node } i \text{ in the network})$$

$$L_{ij} \leq x_{ij} \leq U_{ij} \quad (\text{for each arc in the network})$$

*Flow balance, or flow conservation,
or Kirchhoff equations*

Bound inequalities

Announcements

- Project 2
 - The files will be shared tonight (data, description, fact sheet)
 - Groups are determined, available on Canvas
- Syllabus Adjustment

Date	DoW	Topic	Notes
Apr 8	Wed	Max Flow Problems & Transportation Models	Project 2 Available
Apr 13	Mon	Case Study LP & NM (Locational Marginal Pricing)	
Apr 15	Wed	Intro to Nonlinear Programming (NLP)	
Apr 20	Mon	In-class Office Hour for Projects	
Apr 22	Wed	Case Study Quadratic Programming (Modern Portfolio Theory) NLP: Unconstrained Optimization and SciPy	HW5 Available
Apr 27	Mon	NLP: Constrained Optimization	
Apr 29	Wed	NLP: Algorithmically Solving NLPs & Optimization and Machine Learning	
May 4	Mon	Quiz 3 (All: LP, IP, NM, NLP) <i>(there will be a make-up for students who cannot join due to schedule conflicts)</i>	